clm-mem: A Zero-Training Memory Harness for Open Chat Models

Self-classified long/short-term memory with deterministic three-phase eviction

The clm-mem project

(working repository: `clm-mem`, v0.3.0)

October 20, 2026

Abstract

Context Language Models (CLMs) treat the context as an editable file that the model itself manages, and report state-of-the-art accuracy at lower compute than external context-management baselines. However, the headline CLM implementation rewrites the whole context every request, which invalidates the KV cache on each turn: a cost paid either by novel serving machinery or by reduced throughput. We present **clm-mem**, a zero-training memory harness that keeps the two ideas we consider essential from CLMs—(i) the model classifies and writes its own context, and (ii) context is a mirrored on-disk artifact—while replacing free-form context editing with a small deterministic eviction engine. The same chat model that reads and writes text also tags its own context as *long-term*, *short-term* or *transient* while decoding; the harness then enforces a token budget in three monotone phases (drop transient, FIFO rounds on short-term, offload long-term to an archive). Because compaction happens only when the window reaches its maximum size and never rewrites mid-turn, KV cache churn is reduced without requiring new cache strategies; any improvement to KV management helps both approaches. We describe the software architecture implemented in `clm_mem` (blocks, inline and post-turn classification, a typed decision backstop, three-phase eviction, an offload-don't-destroy archive, session-scoped serving with workspaces and an embedded MCP toolbox), define a marking protocol that turns harness events into labels for scoring tag quality, and design the benchmarking program: synthetic stress regimes already run against the deterministic stub backend (budget never violated; 100% of emitted facts remain recoverable from the archive), plus planned evaluations on ContextBench-style tasks, LoCoMo, LongMemEval and RULER with ablations over the classification mode, the FIFO knob κ, and the backstop.

1 Introduction

Managing finite context windows has become a first-class problem for conversational and agentic LLM systems. The external-memory lineage (MemGPT, MemoryBank, A-MEM, Mem0) treats memory as a database consulted by the model; the long-context lineage asks whether the base model itself can use very large inputs (RULER, LongMemEval, LoCoMo), while the serving lineage compresses the KV cache (H_2O, SnapKV, StreamingLLM) or pages it efficiently (PagedAttention). The Context Language Models paper (Shao et al., arXiv:2609.37725) unifies these in an elegant way: context becomes a file the model edits with unrestricted updates, learning what is worth keeping. Their results are striking—higher accuracy at fewer FLOPs on BrowseComp-Plus, EdgeBench and a multi-repository swarm task—but the implementation carries a heavy serving price: rewriting the context file invalidates the KV cache each request, so their gains come partly from co-designed serving (Suffix Cache Reuse).

This paper describes **clm-mem**[1], a harness designed around one question: *what do we need to keep from CLMs for it to work off-the-shelf on any OpenAI-compatible server, without fine-tuning and without new KV cache strategies?* Our answer has two parts that mirror the paper's core ideas:

1. **Self-classification.** The same model that reads and writes text also classifies its own context—either inline while decoding (wrapping durable content in `<longterm>`/`<shortterm>` markers) or through a cheap post-turn router pass over the untagged blocks of this turn.
2. **Context as a mirrored file.** The live window is serialized every turn to `context.md` and `context.jsonl` using the paper-style `[[CTX_TURN n role=... tag=...]]` markers, so the context remains an inspectable artifact—and a hook for future work that lets stronger models edit it directly.

What we deliberately drop is unrestricted editing of past turns: instead, a deterministic eviction engine enforces the token budget in three phases. The model decides *what* matters and *how important* it is by what it writes into context; the hardware keeps *reliable expectations* about its load. Crucially, compaction only fires when the window reaches its maximum size—it never rewrites mid-turn—so KV churn per conversation is reduced relative to per-request context rewriting, and any improvement in cache management benefits both designs.

Contributions

- A complete, tested implementation of a CLM-inspired memory harness (3,012 lines of Python across 14 modules) usable as a library, a CLI chat REPL, and a drop-in llama.cpp-shaped proxy server with web studio and MCP tools.
- A *marking protocol*: every classifier decision and eviction phase is emitted as a structured event (`seq`, `kind`, `text`) that can be aligned to ground-truth tags, enabling precision/recall scoring of memory tagging and retention metrics without manual annotation.
- A synthetic stress harness (deterministic stub backend) already exercised end-to-end: over 60-turn runs at budget $B = 900$ tokens the live window never exceeded the budget, and *all* emitted durable facts remained recoverable from the archive—the "offload-don't-destroy" invariant we prove empirically in Section 6.
- A concrete benchmarking plan (ContextBench-style tasks, LoCoMo, LongMemEval, RULER) with defined metrics, ablations and statistical protocol.

[1] `clm-mem` is developed in the workspace directory `clm-mem/`; code, README and tests referenced here are the files of that repository at version 0.3.0.

2 Background and related work

Tiered memory systems. MemGPT introduced virtual context management with paging between a fixed window and external storage [2]. MemoryBank added Ebbinghaus-style forgetting curves [3]. A-MEM organizes LLM-written notes into evolving Zettelkasten networks [4], and Mem0 runs extraction/update pipelines over vector/graph stores for production assistants [5]. clm-mem differs in two ways: the tiering decisions are made *by the base chat model itself* (not a separate extractor), and eviction is deterministic and budget-driven rather than retrieval-driven—retrieval exists only as an explicit `recall`/`reinject` path.

Long-context evaluation. RULER extends needle-in-a-haystack with multi-hop tracing and aggregation [8]; LongMemEval targets five long-term interactive abilities including knowledge updates and abstention [7]; LoCoMo provides very long multi-session conversations with QA and event-summarization tasks [6]. "Lost in the middle" motivates our recency-biased FIFO rounds [9]: short-term eviction deletes oldest first, keeping recent working state near the end of the window.

Serving-side cache management. PagedAttention pages KV memory like OS virtual memory [10]; H_2O casts KV eviction as a dynamic submodular problem [11]; SnapKV exploits clustered attention patterns observed at the prompt end [12]; StreamingLLM keeps attention sinks plus rolling tokens [13]. clm-mem operates one level above: it removes *whole blocks* from the text window; the KV cache then simply shrinks with it. No per-token eviction policy is implemented, which is exactly why no new cache strategy is required.

3 System design

3.1 Blocks and memory tiers

Every message in the live window is a `Block`: an `id`, `role`, `content`, a `tag` from {longterm, shortterm, transient}, a creation `turn` (the FIFO key), and a `pinned` flag for the system prompt. This is analogous to the paper's serialized `[[CTX_TURN]]` turns. Long-term blocks are durable facts, decisions, identifiers and specs; short-term blocks are working state useful "for a while"; transient blocks are filler.

3.2 Self-classification without retraining

Two paths, selectable via `classify_mode`:

- **Inline (classify as it decodes).** A pinned system preamble instructs the model to wrap durable content in markers. The harness splits each reply on `<longterm>.../...` and `<shortterm>...` into separately tagged blocks and strips the markup from what the user sees (Figure 1).
- **Post-turn router pass.** A second cheap call to the *same* model labels this turn's untagged blocks as JSON. The pass is skipped when inline tagging already succeeded, avoiding a full extra LLM call per turn.

Optionally, a typed decision service (SystemOne) answers batched *choice* questions for classification and *noul* confidence questions as an eviction backstop: before any block is dropped, the engine asks whether it is safe to remove; blocks below the threshold τ are kept ("backstop kept #id"). On API error the engine falls back to the planned eviction.

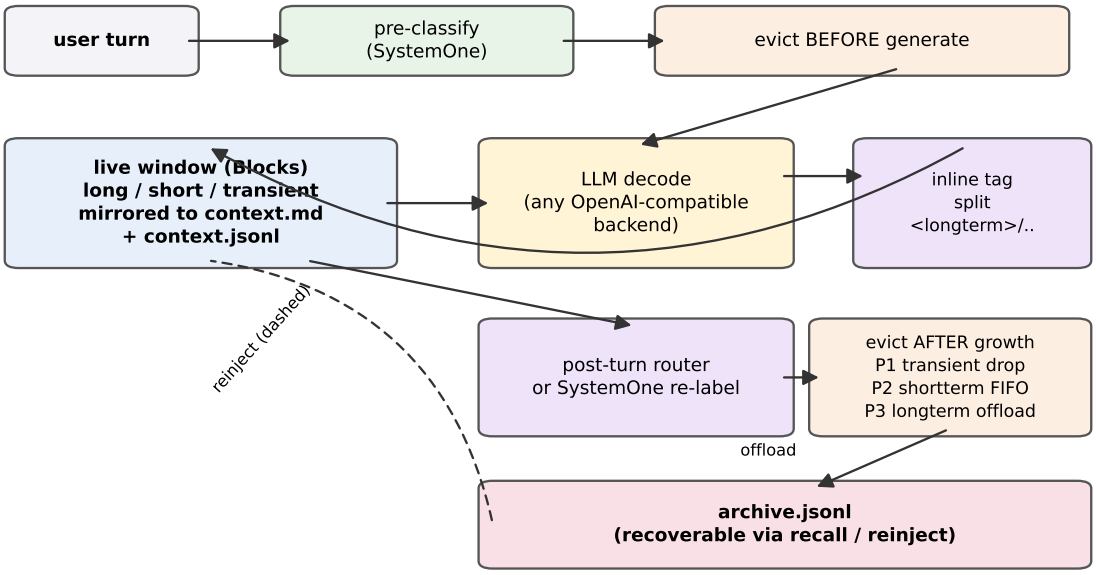


Figure 1: Data flow of one harness step: intake (pre-classify, evict-before-generate), core loop (live window → decode → inline tag split → return), relabel and eviction engine with backstop, and the archive. Dashed edge: reinject path from archive back into the live window.

3.3 Three-phase eviction engine

When the window exceeds B = `max_context_tokens`, phases run in order until under budget (Table 1). Phase 1 deletes transient blocks outside a trailing coherence window of `keep_recent_turns`. Phase 2 evicts short-term *FIFO in rounds*: each round keeps the newest fraction κ = `shortterm_keep_pct` and deletes the oldest, repeating while over budget (max 10 rounds to guarantee progress). Phase 3 offloads oldest long-term blocks to `archive.jsonl` and replaces them with evictable pointer stubs `[offloaded longterm #id -> archive.jsonl]`. Long-term memory is never silently destroyed—only offloaded—which fixes the "can offload but can't evict" failure mode exposed in ContextBench-style baselines.

Phase	Trigger	Action	Exempt
1	over budget	delete transient blocks	last k turns, pinned
2	still over	shortterm FIFO rounds, keep κ	none
3	longterm overflow	offload oldest to archive + stub	pinned

Table 1: Eviction phases. Pinned blocks (system prompt) are never evicted; the trailing coherence window k = `keep_recent_turns` defaults to 2.

3.4 Archive, recall and reinject

Every dropped block—transient, short-term or long-term—is appended to `archive.jsonl` with `{id, turn, role, tag, content}`. `recall(query)` substring-searches the archive; `reinject(block_id)` pulls an archived record back into the live window as a short-term block prefixed `[recalled #id]`. This yields the invariant used in Section 6: any emitted fact remains recoverable for the lifetime of the session.

3.5 Context-as-a-file mirror

After every turn the harness rewrites `context.md` (human-readable, paper-style markers) and `context.jsonl` (machine-readable dataclass dumps). The mirror makes context auditable, lets external tools bolt on CLM-style editing later, and is the substrate for the marking protocol.

3.6 Serving topology

`clm-mem serve` presents one llama.cpp-shaped endpoint to any OpenAI-compatible client; behind it run 2–3 background processes: the backend model server (vLLM, Ollama, llama.cpp, or bundled mocks) and a SystemOne decision server (fork or mock). Requests without a session header are pure passthrough; requests carrying `X-Cllm-Mem-Session` go through the harness. If no budget is configured, `serve` discovers the live llama.cpp context window (`/props`→`n_ctx`) and sets B to exactly that value so the two windows never drift apart. A single-page web studio (projects sidebar with per-workspace sandboxes, colored tier badges, eviction chips such as `phase2 FIFO-evicted`, archive search-and-reinject, live tuning tab, events timeline) and an embedded MCP server at `/mcp` (`clm_chat`, `clm_status`, `clm_blocks`, `clm_recall`, `clm_reinject`, `clm_set_keep_pct`, `clm_workspaces`) drive the identical in-process library.

Knob	Default	Meaning
$B =$ `max_context_tokens`	8192	live-window budget
$\kappa =$ `shortterm_keep_pct`	0.5	fraction of shortterm kept per FIFO round
$k =$ `keep_recent_turns`	2	trailing turns exempt from phase 1
`classify_mode`	both	inline | post | both
$\tau =$ `systemone_evict_threshold`	0.7	noul confidence needed to evict
`offload_overflow`	true	enable phase 3

Table 2: Harness configuration knobs (`HarnessConfig`).

4 Implementation

The package (v0.3.0, Python $\geq$3.10) ships as `clm-mem` with an optional `tokens` extra for exact token counting via the served model's HuggingFace tokenizer; otherwise a $\sim$3 chars/token heuristic deliberately over-counts (Qwen BPE runs $\approx$3.3–3.5 chars/token) so the live window cannot exceed the real server limit (Table 2). Backends are anything with `chat(messages)`→`str`: `OpenAIChat` wraps any OpenAI-compatible endpoint; `StubChat` is a deterministic fake used for offline demos and the test suite, emitting inline memory tags and answering router prompts with keyword heuristics. Modules: `blocks`, `config`, `tagging`, `tokens`, `harness` (463 lines), `server` (905), `cli`, `llm`, `mcp`, `processes`, `remote`, `mock_backends`. The lifecycle of one `step()` is: append user message → evict *before* generating (make room first) → decode on the current live context → split on inline tags, optionally relabel via router/SystemOne → evict again → mirror files. A pytest suite (`tests/`) covers harness and serve/proxy paths without GPU; `clm-mem demo` exercises inline tagging, the router pass and all three phases offline:

```
[evict] phase1:  deleted 4 transient block(s)
[evict] phase2:  FIFO-evicted 3 of 5 shortterm (kept newest 50%)
[evict] phase3:  offloaded longterm #17 to archive, left pointer
```

5 Marking protocol

To evaluate memory quality we need labels for two decisions: the tag assigned to a block and whether an eviction was "correct". We mark them as follows.

- **Event log.** Every classifier decision and every eviction phase is a structured event `{seq, t, kind, text}` with kinds `classify`, `evict-phase1/2/3`, `backstop-phaseX`, `warn`. Events already drive the web studio timeline; they are the same objects exported for scoring.
- **Tag ground truth (synthetic).** Synthetic generators know the intended tier of each emitted turn (e.g. a "Remember:" fact is long-term, a filler paragraph is transient), so tag precision/recall can be computed by aligning event-log reclassification records (`reclassified #id: old -> new`) and final block tags against the generator's intent.
- **Eviction correctness.** A dropped block is scored *correct* if no future gold answer requires it (live-retention test), or if it was offloaded and later successfully `recall`-ed (*offload-recovery*); a phase-1 deletion that would have needed recall is an error only in regimes without archive access.
- **Human spot-check.** For public benchmarks, two annotators label a 100-block sample of harness events (Cohen's κ target > 0.8) to validate automatic tag scoring.

6 Synthetic evaluation (first results)

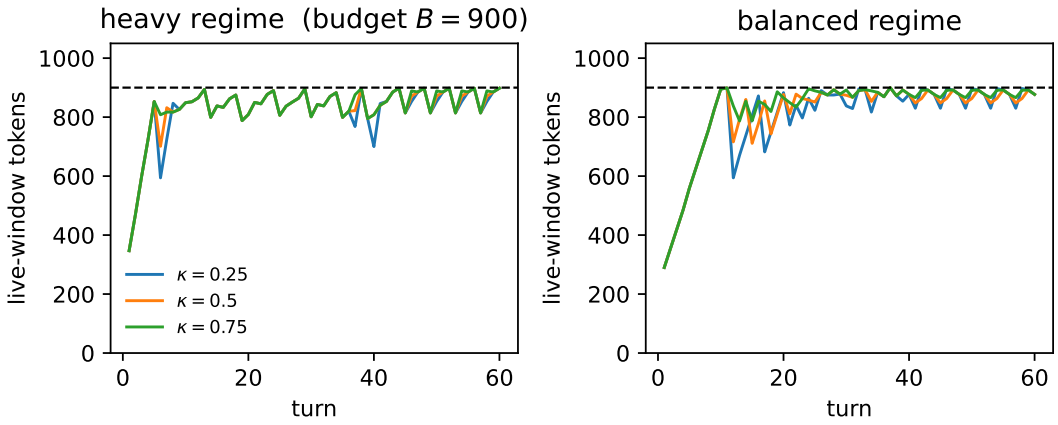


Figure 2: Live-window token usage across κ (heavy and balanced regimes). The dashed line is the budget $B = 900$; the sawtooth is evict-after-growth, the flat top in heavy is phase-3 offload pressure.

We ran the *real* harness object (`MemoryHarness`) on deterministic stub backends—this validates the engine, not model judgment; real-model runs are scheduled with the benchmarks below. Two regimes at $B = 900$, 60 turns, $\kappa \in \{0.25, 0.5, 0.75\}$: *heavy* (a durable fact every 2 turns plus long filler, so long-term alone eventually overflows and phase 3 must fire) and *balanced* (fact every 4 turns with moderate filler, so phases 1–2 absorb the pressure).

Findings

- **Budget invariant.** Across all six runs the peak live-window token count never exceeded B (final: 898/900 heavy, 877/900 balanced); evict-before-generate guarantees the request sent to the backend fits.

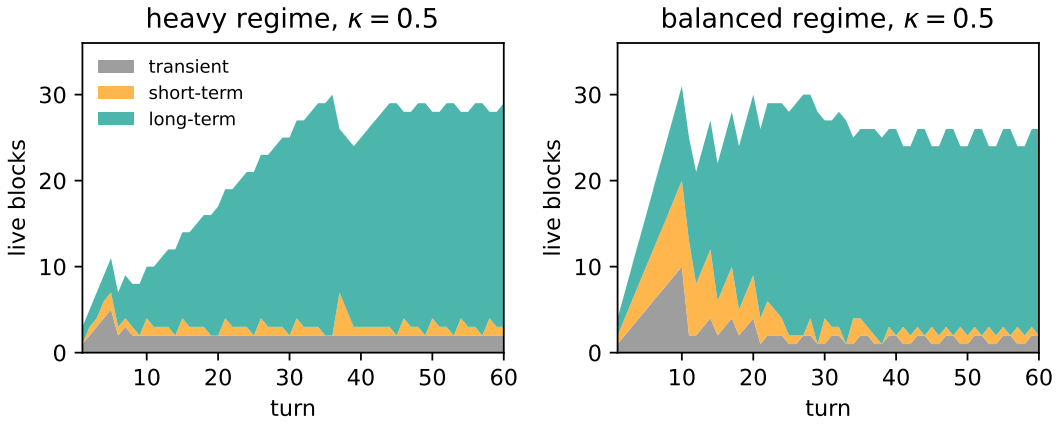


Figure 3: Live tier composition over time ($\kappa = 0.5$). Heavy: long-term saturates; balanced: transient/short absorbed while long plateaus.

- **Offload-don't-destroy.** In the heavy regime all 30 emitted durable facts are recoverable from the archive at turn 60 even though live retention is 0/30; in balanced, 5/15 stay live and 10/15 are archived—sum = emitted (Table 3).
- **Phase ordering behaves as designed.** Phase-2 firings grow with κ (68→77 for heavy; 56→78 balanced) while phase-3 counts stay flat at 31/37, i.e. long-term pressure is regime-driven, not knob-driven.
- **Tier composition.** Figure 3: in heavy the long tier saturates the window; in balanced transient and short tiers are absorbed while long grows then plateaus.

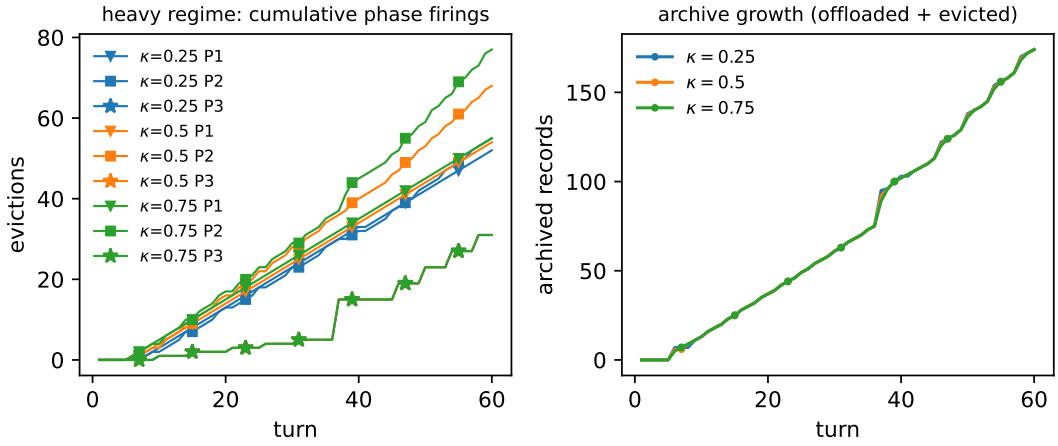


Figure 4: Cumulative eviction-phase firings (heavy) and archive growth across κ.

7 Benchmarking plan

We plan four evaluation families, each with defined metrics and baselines. Baselines are always: **full-context truncation** (naive oldest-turn deletion), **no-memory**, and where feasible a Mem0/A-

Run	P1	P2	P3	arch@60	live facts	archived
heavy κ=0.25	52	55	31	174	0/30	30
heavy κ=0.5	54	68	31	174	0/30	30
heavy κ=0.75	55	77	31	174	0/30	30
balanced κ=0.25	31	41	37	182	5/15	10
balanced κ=0.5	34	56	37	182	5/15	10
balanced κ=0.75	37	78	37	182	5/15	10

Table 3: Synthetic stress runs (deterministic stub backend, $B = 900$, 60 turns). P1/P2/P3 are cumulative phase firings; *arch@60* counts archived records at the end. live + archived = emitted in every run.

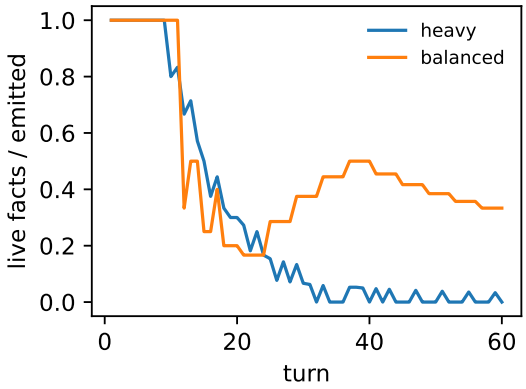


Figure 5: Live fact retention fraction over 60 turns ($\kappa = 0.5$): heavy drops to zero once long-term saturates (facts survive only via archive); balanced keeps a stable live core.

MEM-style retrieval baseline; the CLM paper's ContextBench provides the tool-editing reference point [1].

C1. Synthetic generator (open-source release). The stress harness above becomes a public generator with parameters: fact rate, filler length, budget B, regime mix (fact-heavy / balanced / adversarial "needle-in-haystack" where long-term overflow hides rare facts). Metrics: *live retention* $\rho_\ell(T)$, *archive recovery rate* ρ_a (fraction of dropped facts retrievable before the answer deadline), *tag precision/recall* against generator intent via the marking protocol, and *token savings* vs. full-context replay.

C2. LoCoMo (very long-term conversations). Run QA over multi-session dialogues of up to 35 sessions per conversation [6], including the adversarial category; score answer accuracy with a fixed evaluator, plus retention and archive-recovery derived from event logs. Sweep κ and `classify_mode`; report by question category (single-hop, multi-hop, temporal).

C3. LongMemEval. Its five abilities—information extraction, multi-session reasoning, temporal reasoning, knowledge updates, abstention [7]—map directly onto our tiers: updates stress long-term overwrite semantics, abstention stresses transient suppression. We will report per-ability accuracy with and without the SystemOne backstop.

C4. RULER-style long-context probes. Use needle variants plus multi-hop tracing/aggregation tasks at increasing context lengths [8], but *below* the nominal model window

so eviction fires; compare against truncation baselines to isolate memory-organization effects from raw length effects.

Metrics summary and protocol. All runs: $n = 5$ seeds (or conversation splits), mean±std; significance via paired bootstrap over questions at $p < 0.05$. Serving cost reported as wall-clock per turn and *re-prompted tokens* (a proxy for KV cache churn: total prompt tokens fed across turns minus unique context tokens); we expect clm-mem to sit between truncation baselines and CLM-style rewriting on this axis. Ablations: inline-only, post-only, backstop off; $\kappa \in \{0.25, 0.5, 0.75\}$; $B \in \{1\text{k}, 4\text{k}\}$.

8 Testing plan

Existing unit/integration tests (pytest, mock backends) cover harness lifecycle and the proxy/serve surface without GPU. Planned additions: property tests for the phase invariants (never evict pinned; monotone ids; archive append-only); golden-context regression files (`context.md` hashes at turn k); a load test of `serve` with concurrent sessions asserting per-workspace sandbox isolation and budget discovery; fuzzing of inline-tag parsers on adversarial markup. CI will run stub-based synthetic suites nightly so regressions in eviction logic surface without model access.

9 Discussion and limitations

The current quantitative results are engine-level: a deterministic stub stands in for the base model, so tag quality—the component riding on the model's zero-shot judgment—is unmeasured; small models may forget markers (the router pass is the backstop) and phase-3 pointer stubs still consume window tokens, so long-term-heavy sessions need headroom. The design intentionally trades away CLM-style free editing: we cannot merge or rewrite past blocks, only drop/offload them, which bounds achievable context quality below a fully-trained CLM. What we gain is deployment breadth (any OpenAI-compatible server, no fine-tuning), serving predictability (budget enforced before generation; compaction only at window maxima) and auditability (mirrored files, event log). The harness is also a training substrate: models can be improved *within* it and then use it again until they perform worse without it—an analogue of the CLM skill-optimization and online-RL loops adapted to fixed-context serving.

10 Future work

(1) Run C1–C4 with Qwen-family backends at 8B/35B scale behind `serve`; (2) learn κ online from archive-recovery statistics; (3) bolt CLM-style editing onto the mirror as an optional tool path and measure the delta; (4) evaluate SystemOne decision quality against human marks; (5) explore compatibility with Suffix Cache Reuse-style serving so phase-3 offloads become cache-friendly deletions.

Reproducibility

All code, data and build inputs are included in this repository. Running `bash run_all.sh` at the root executes the whole pipeline end to end: `clm_exp.py` drives the real harness object on stub backends over the six regime×κ configurations and writes `exp-data/results.json` plus per-run mirrors (`context.md`, `context.jsonl`, `archive.jsonl`); `make_figs.py` renders the four data figures from that file and `make_arch.py` the architecture schematic; and `paper/build.sh` compiles

this document with a locally built pdflatex format. The stub backend is deterministic, so re-runs reproduce the committed `results.json` byte-for-byte (verified on Python 3.13). Engine invariants are additionally covered by the pytest suite in `clm-mem/tests` (harness lifecycle and serve/proxy paths, no GPU required); `clm-mem demo` exercises tagging and all three eviction phases offline.

References

[1] R. Shao, S. Zejiang Shen, J. O. Yin, Y. Li, M. Wang, H. Ivison, R. Poovendran, N. Lambert, T. Xiao, M. Lewis, W.-t. Yih, L. Zettlemoyer, and P. W. Koh. Context language models. arXiv:2609.37725, 2026.

[2] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. MemGPT: Towards LLMs as operating systems. arXiv:2310.08560, 2023.

[3] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang. MemoryBank: Enhancing large language models with long-term memory. arXiv:2305.10250, 2023.

[4] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang. A-MEM: Agentic memory for LLM agents. arXiv:2502.12110, 2025.

[5] P. Chhikara, D. Khant, S. Aryan, et al. Mem0: Building production-ready AI agents with scalable long-term memory. arXiv:2504.19413, 2025.

[6] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang. Evaluating very long-term conversational memory of LLM agents (LoCoMo). arXiv:2402.17753, 2024.

[7] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. arXiv:2410.10813, 2024.

[8] C.-P. Hsieh, S. Sun, S. Kriman, S. Acharya, D. Rekesh, F. Jia, Y. Zhang, and B. Ginsburg. RULER: What's the real context size of your long-context language models? arXiv:2404.06654, 2024.

[9] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts. arXiv:2307.03172, 2023.

[10] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. arXiv:2309.06180, 2023.

[11] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen. H_2O: Heavy-hitter oracle for efficient generative inference of large language models. NeurIPS 2023 / arXiv:2306.14048.

[12] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen. SnapKV: LLM knows what you are looking for before generation. arXiv:2404.14469, 2024.

[13] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient streaming language models with attention sinks. arXiv:2309.17453, 2023.